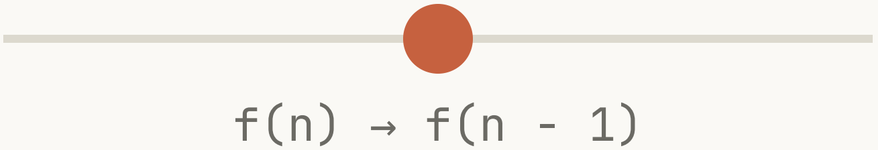
LESSON 23

大事 化小



递归入门：函数调用自己

学完你能让函数调用自己，把一件大事拆成一样的小事，一层层做完。

像打开套娃

打开一个套娃

里面还有一个

打开它

再小一号

打开到最小的那个

打不开，停

每次做的事都一样：打开。只是套娃一次比一次小，小到最后一个就停下。

函数也能这样：在函数里调用自己，每次处理小一点的问题，小到一眼能答就停。这叫递归。

函数调用自己：倒数发射

countdown.py

```python
def countdown(n):
    if n == 0:
        print("发射！")
    else:
        print(n)
        countdown(n - 1)
countdown(3)
```

到了 countdown(0)，n == 0 成立，输出“发射！”，不再调用自己，停下。

3

2

1

发射！

猜一猜：先调用自己，再输出，顺序会怎样?

up.py

```
1 def up(n):
2     if n > 0:
3         up(n - 1)
4         print(n)
5 up(3)
```

1
2
3

up(3) 要先等 up(2) 全部做完，才轮到 print(3)。up(2) 又要先等 up(1)……
所以最里面的 up(1) 最先输出。到了 up(0)，n > 0 不成立，什么也不做。

小测：改成 countdown(0)，输出什么？

1 发射！

2 先 0，再 发射！

3 什么也不输出

4 报错

答对了！一进去 n == 0 就成立，直接输出“发射！”，不再调用自己。

递归要有两件事

出口：什么时候停

```
if n == 0:
```

小到一眼能答，就不再调用自己

变小：每次近一步

```
countdown(n - 1)
```

每调用一次，离出口近一点

少了哪一件，函数都会一直调用自己。调用的层数太多，Python 就停下，报 RecursionError：递归太深了。

写递归，先想好这两件事。

例题：从 1 加到 n

total.py

```python
def total(n):
    if n == 1:
        return 1
    return n + total(n - 1)
print(total(int(input())))
```

结果一层层交回去：2 + 1 是 3，3 + 3 是 6，4 + 6 是 10。最后输出 10。

total(4)	4 + total(3)
total(3)	3 + total(2)
total(2)	2 + total(1)
total(1)	1

1 → 3 → 6 → 10

小测：例题里输入 5，输出多少？

1 5

2 10

3 15

4 120

答对了！5 + total(4)，total(4) 是 10，合起来是 15。

动手改一改

把加法换成乘法。

从 1 乘到 n

读入 n（1 到 10 的整数），输出 1 × 2 × 3 × … × n。这个数叫 n 的阶乘。在例题上改，想一想出口还是 1 吗。

输入 4	输入 1	输入 10
24	**1**	**3628800**

Python 里乘号写 *。

这节课常见的三种错误

划线的写法只阅读，不要照抄

没写出口	**RecursionError**	先想好什么时候停
~~`countdown(n)`~~	**RecursionError**	每次要变小：`n - 1`
忘了写 `return`	**TypeError**	交回 `None`，拿去加数就报错

看到 `RecursionError`，先查两件事：有没有出口，每次有没有变小。

找错：程序报 RecursionError，停不下来了

bug.py（只阅读）

```
1 def total(n):
2     if n == 1:
3         return 1
4     return n + total(n)
5 print(total(4))
```

想算 1 加到 4，却一直调用下去。
点出让它停不下来的那一行。

点代码里的一行

自己挑战

数字加起来

读入一个正整数 n（1 到 999999999），用递归把它的每一位数字加起来，输出结果。

比如 2026：2 + 0 + 2 + 6 = 10。不许把 n 变成字符串，只用数和递归。

输入

```
2026
```

输出

```
10
```

试试 7 和 10。先自己写，卡住了再翻下一页。

卡住了？一次看一条提示

提示 1　只有一位数时（n < 10），各位加起来就是它自己。这就是出口。

提示 2　n % 10 是个位（第 3 课学过），n // 10 是去掉个位以后剩下的数（第 7 课用过）。

提示 3　各位之和 = 个位 + 剩下那个数的各位之和。后面这一半，就是调用自己，n 每次少一位。

参考答案：数字加起来

digit_sum.py

```
def digit_sum(n):
    if n < 10:
        return n
    return n % 10 + digit_sum(n // 10)
print(digit_sum(int(input())))
```

个位是 n % 10，剩下的 n // 10 交给自己去加；只剩一位时直接还 n。

造一个你的递归小程序

让函数调用自己。做什么，你自己定。

先做到

改一改倒数，换成你自己的话，比如“还有 3 天放假”。

选你喜欢的

递归打印星号三角形、算 2 的 n 次方、从 n 倒着数到 1 再数回来。

还想更厉害

先变小再变大的沙漏图形；或者兔子数列 1 1 2 3 5 8。

没有标准答案。先说清它要做什么，再试两组输入，一组试边界。

和同桌换着玩

1

换着运行

运行同桌的程序，先猜输出，再看对不对

2

说一个喜欢的

告诉同桌，哪里拆得最巧

3

提一个想法

出口和变小都有吗？输入 `0` 会怎样？

听完再改一版，让程序更稳。

说得清吗?

Q1	什么是递归?	函数调用自己，每次处理小一点的问题。
Q2	写递归要有哪两件事?	出口和变小。
Q3	`total(3)` 怎么算出 6?	`3 + total(2)`，`total(2)` 是 `2 + total(1)`，`total(1)` 是 `1`，再一层层交回。

速查卡

`countdown(n - 1)`	函数里调用自己，每次变小
`if n == 0:`	出口：小到一眼能答就停
`return n + total(n - 1)`	用小一号的结果算出大的
先调用，后输出	最里面那层最先输出
`RecursionError`	查出口，查变小

这些后面再学

兔子数列

直接递归会把同一个数算很多遍，很慢

记住算过的

把算过的结果存进字典，下次直接拿

汉诺塔

一层里调用自己两次，把一摞盘子搬到另一根柱子

今天学会了把大事拆成一样的小事。下节课学排序，看看怎样把一堆数排整齐。

谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 python-for-youth

下一课：冒泡排序